A demo data room in one minute — step by step

Audience: trainers, presenters and test users who need a complete demo portfolio with a story that can be told — no Python knowledge required. Duration: one minute to the first data room, about 30 minutes for the whole tutorial. Prerequisites: SituationReport (beta build with test data generator 3), a browser. Nothing gets installed; the generator needs neither network nor model. At the end: you generate a data room at the click of a button, choose story and structure, read the series of stands and know which file is there for what.

This tutorial is also available as PDF: DE / EN.

1. What this is about — and what it is not

A data room is a folder holding everything the suite can read for a portfolio: Jira exports per ART, the workbooks derived from them, registers per solution (risks, dependencies, NFRs, capabilities, decisions, SLOs, delivery, flow problems, themes, solution areas, decision points), configurations, a series of frozen stands with the delta briefings between them, the providers' raw data and a README that tells the story. The portfolio manager reads it exactly as it reads a real data room.

The generator produces a demo data room: the same shape, invented teams, a chosen story. Three things always hold:

- No persons. Owners and reporters are teams, ARTs and value streams.
- Reproducible. Same seed, same structure, same story, same reference date — same

data room.

- The registers are the truth. Raw data and stands are derived from them, never the

other way round.

What the demo data room is not: a picture of a real company. The numbers are plausible but invented; the story is a template, not an observation.

2. One minute: the browser UI

2.1 Start

```
python -m testdata_generator --web
```

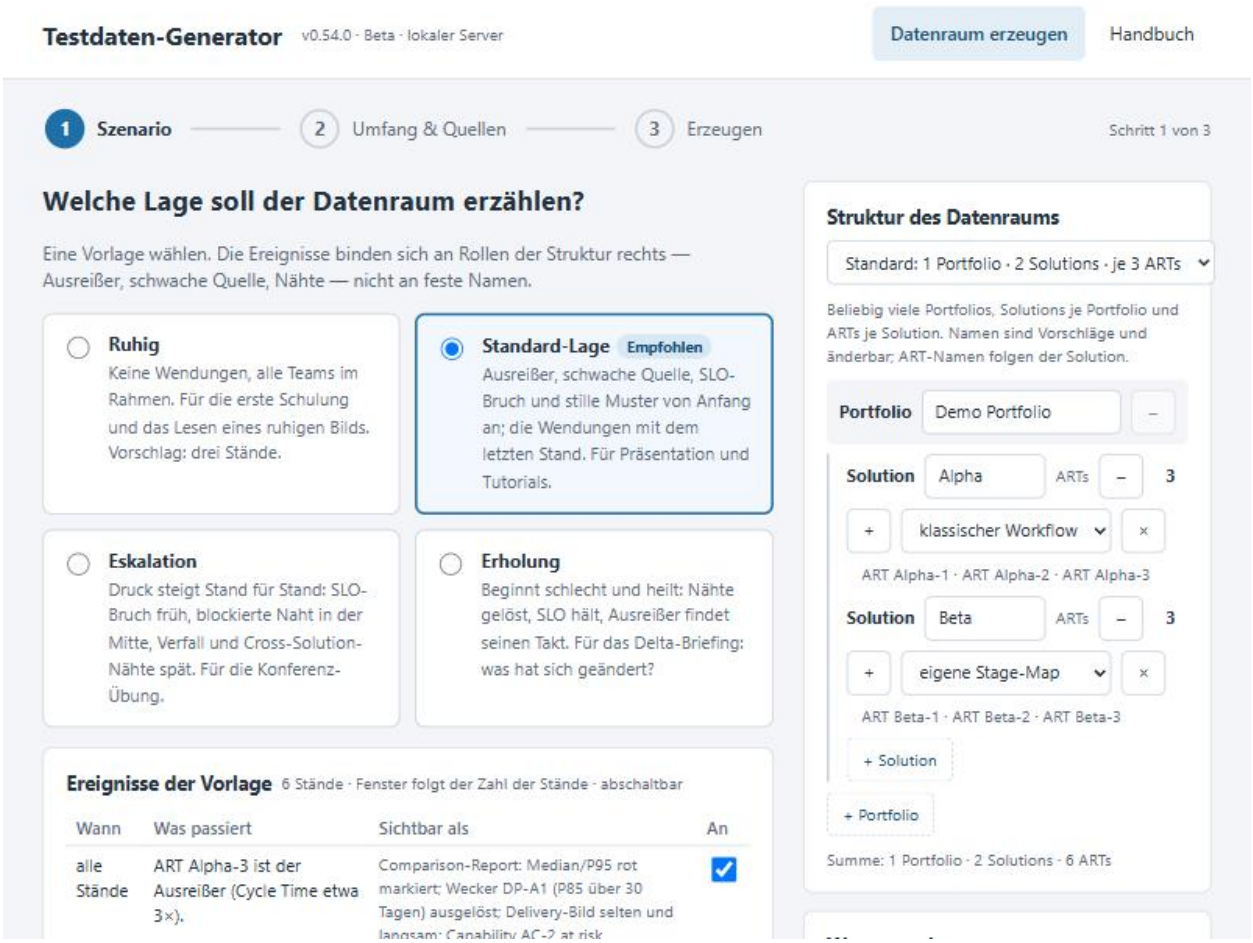
Your browser opens at an address like `http://127.0.0.1:51347/3f9a…/`. The page runs on a small server only on your machine (address 127.0.0.1, random port, random key in the link). Whoever lacks the link gets a 403 page; nothing is reachable from the network. End it with Ctrl+C in the window you started from.

The UI starts in your launcher's language (German, English, Romanian, Portuguese or French) and can be switched at the top right; `--lang de` forces a language. The generated texts — story, README, delta briefings — follow the same language. With `--no-browser` the generator only prints the link; `--port 8765` fixes the port.

Without a command line: start the card "Data room (browser)" in the launcher (`SituationReport.bat`); the portable package also ships `DatenraumBrowser.bat` for it (macOS/Linux: `DatenraumBrowser.command` / `.sh`).

On a corporate machine take the window: `python -m testdata_generator --dataspace`, the launcher card "Data room (window)" or `DatenraumFenster.bat`. The browser path starts a local server that endpoint security software may block and report (it happened in the first test run). The window has the same three steps and the same functions, only without server and browser — everything below applies there too.

2.2 Step 1 — Scenario



Step 1: template, events and structure

On the left you pick the template. "Standard" is recommended: an outlier ART, a weak data source, an SLO breach and the silent patterns from the start; blocked seam, cross-solution pressure, a third solution area, confidence decay, a new risk and updated roadmaps with the last stand. Below it the events of the template with their stand window and a switch — for the first minute leave everything as it is.

On the right is the structure: one portfolio, two solutions, three ARTs each. Leave that too for the first minute. "What you get" shows the numbers that follow from the structure.

Click Continue to scope & sources.

2.3 Step 2 — Scope and sources

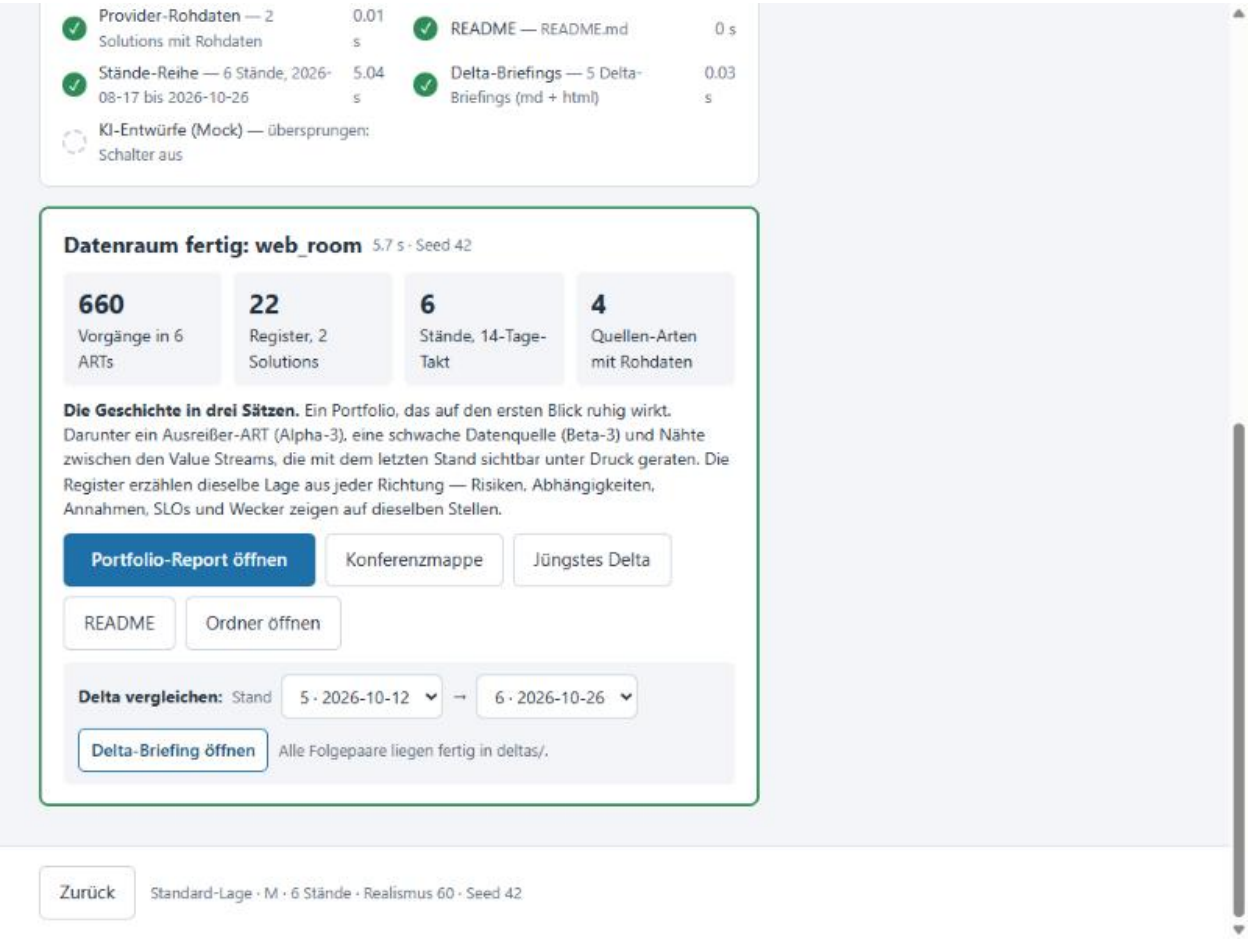
The defaults are enough: six stands two weeks apart, scale M, realism 60, seed 42, last stand today, language of the texts as the UI. On the right you see which sources are always included; the AI drafts switch stays off.

Click Continue to generate.

2.4 Step 3 — Generate

The target folder is suggested (a new folder in your home directory). Change it with "Browse …" or type a path. Then Generate data room.

The progress list shows the nine steps with their duration: structure and story, Jira exports and workbooks, registers, configurations, provider raw data, README, series of stands, delta briefings, AI drafts (skipped, switch off). After a few seconds the result card appears.



Step 3: result card with numbers, story and actions

The four numbers: issues in 6 ARTs, 22 registers in 2 solutions (11 per solution), 6 stands, 4 source kinds with raw data. Below them the story in three sentences. The buttons open the portfolio report, the conference pre-read, the latest delta briefing, the README and the folder; the comparison picker opens the delta between any two stands.

That was the minute. Everything else is reading and varying.

3. The same on the command line

```
python -m testdata_generator --scenario portfolio --output demo --seed 42
```

This produces exactly the data room the UI produces with its defaults. The switches of the three steps are called:

	standard\	escalation\
	standard\	large `or` --structure FILE.json`
	m\	l`

Before building, the command line prints the estimate ("6 Jira exports, 18 workbooks, 22 registers, 6 stands, 5 delta briefings") and warns about large structures; afterwards the result line with issues, registers, stands, deltas and seconds.

4. What the folder holds

```
demo/
■■■ structure.json · story.json   ← structure and story as data
■■■ portfolio.json                ← entry point for the portfolio manager
■■■ solutions/alpha/solution.json ← per solution: config, arts/, registers/
■■■ solutions/beta/…
■■■ workflows/  raw/  pi_config.json
■■■ sources/alpha/ · sources/beta/ ← provider raw data + sources configs
■■■ snapshots/snapshot_01_… to _06_…, stands.json, snapshot_prev/now.json
■■■ deltas/delta_01-02.md/.html … delta_05-06
■■■ README.md                      ← structure, story, stands, sources, commands
```

Read the README first: it lists the solutions with their ARTs, the events of the story with their stand windows, the stands with date and file and the sources with their commands. Whoever passes the folder on passes the explanation with it.

The folder is movable: every path in the configs is relative. Copy it, zip it, put it on another machine — the portfolio manager reads it there just the same.

5. Choosing the story

Four templates, the same structure, different events:

Template	For whom	What the stands show
Calm	the first training	no turning points; the exercise is reading a picture that says nothing dramatic — whoever raises an alarm here has misread
Standard	presentations, tutorials	the turning points with the last stand; before that throughput and alarm movements
Escalation	the conference exercise	SLO breach early, blocked seam mid, decay and cross-solution seams late, new risk last
Recovery	the delta briefing	starts bad and heals: seams resolved, SLO holds, the outlier finds its pace, the data source last

Each of the ten events (E1 outlier to E10 roadmaps) has a stand window given as phases — start, early, mid, late, end — that adapts to the number of stands, and a switch. Switch an event off and the register shows its healthy state: without the outlier the last ART delivers like its sisters, without the SLO breach "Order Sync API" keeps its budget. An event whose prerequisite is off (the confidence decay needs the weak source) stays off with a notice — the UI and `story.json` say so.

Try: template Escalation, then in step 3 set the comparison picker to 3 → 4. The delta shows how dependency AD-1 becomes blocked.

6. Changing the structure

The structure is data (`structure.json`), not code. In the UI you change it on the right in step 1: add solutions, count ARTs, choose the workflow, create portfolios. Names are suggested (Alpha, Beta, Gamma …) and editable; ART names follow the solution.

The story binds to positions, not names: the outlier is the last ART of the first solution, the weak source the last ART of the second. What the structure cannot provide is dropped with a notice: a single solution has no weak source and no cross-solution seam; a one-ART solution has no outlier; a third solution carries only the seed-generated base population.

On the command line:

```
python -m testdata_generator --scenario portfolio --arts 3,4s,1 --output demo-3
python -m testdata_generator --scenario portfolio --structure large --output demo-5
```

With several portfolios each gets a sub-folder with its own `portfolio.json` and its own stands; the root holds the whole `structure.json` and an index README. From 30 ARTs on the estimate warns about the duration but refuses nothing.

7. Stands and delta briefings

Every stand is a snapshot (`snapshots/snapshot_<nn>_<date>.json`), 14 days from the next, the last one on the reference date. `stands.json` lists them; `snapshot_prev.json` and `snapshot_now.json` are copies of the last two so that everything expecting prev/now keeps working.

Every consecutive pair is ready under `deltas/` as Markdown and HTML. For another pair:

```
python -m portfolio --delta demo/snapshots/snapshot_02_*.json demo/snapshots/snapshot_05_*.json --output delt
```

A training can start at stand 1 and uncover the situation stand by stand: which number moves, which register reports first, when the alarm should have rung.

Earlier stands are the same story two weeks younger: fewer issues and the registers in the state the template prescribes for that stand. The stands are aggregate series of one story, not an issue time series with stable keys — for delta briefings, alarms and the spoken briefing exactly what is needed.

8. Provider raw data: showing the chain

Under `sources/<solution>/` lies what the systems would deliver — Prometheus responses, GitHub deployments, pull requests and incidents (first solution) or GitLab DORA daily values (second solution), SonarQube measures, an SLO CSV — as replay files in each system's own format, plus the configs:

```
python -m sources fetch --kind slo --config demo/sources/alpha/slo_sources.json --output fetched_slo.json
python -m sources fetch --kind dora --config demo/sources/alpha/dora_sources.json --output fetched_dora.json
```

Nothing goes to the network: the config carries `"replay"`, and the provider reads the recorded responses exactly as it would read them live. Compare `fetched_slo.json` with `demo/solutions/alpha/registers/slo.json` — the same values. Only GitHub derives DORA from counts and comes close only approximately for small units (one failed deployment out of five is 20 percent, not 14); the README says so.

9. Practising AI drafts (switch)

Switch on AI drafts in step 2 (or `--ai-drafts`). The data room gets a folder `ai_drafts/` — deliberately separate from the rest, because these are results, not inputs:

- `einseiter.html` with the spoken-briefing draft beside it (mock provider, no model),
- three answers to questions about the picture (`frage_1.ask.md` …),
- an exported prompt bundle and `uebung_antwort_falsch.md` with two numbers that are in

no contract,

- `uebung_README.md` with the exercise:

```
python -m llm check demo/ai_drafts/uebung_antwort_falsch.md --bundle demo/ai_drafts/prompt_d3_question_*.md
```

Expected: the numbers guard rejects both numbers, nothing is stored as a draft. Every draft carries the AI marker, none is approved; practise approval with `python -m llm approve <file>`. The evidence log is `ai_drafts/llm_audit.jsonl`.

10. Continuing, and what can go wrong

Appending a stand:

```
python -m testdata_generator --scenario portfolio --continue demo --to-date 2027-01-15
```

The Jira exports are continued to the new date, the workbooks recomputed, the new stand becomes stand 7 and the delta 6 → 7 is rendered; the registers stay.

What you see	What it means	What you do
"Folder is not empty"	the target folder already holds files	choose another folder or generate into it deliberately (same names are overwritten)
403 page in the browser	the link without its key, or an old link	take the link from the start window; every start has a new key
page does not load	a proxy or policy blocks 127.0.0.1	fallback: `--dataspace` or `DatenraumFenster.bat` — the same steps as a window; or the command line from section 3
"No folder dialog available"	tkinter is missing on this machine	type the path into the field
warning "large structure"	from 30 ARTs or 5 portfolios at scale L	wait, or choose scale S
notice "only one solution"	the story has no second solution	as intended; add a second solution when you need the weak source

11. What to take away

- A data room is a folder with everything the suite reads — demo and real have the same shape.
- One minute: `--web`, keep the template, continue, continue, generate.
- Story and structure are data (`story.json`, `structure.json`); events bind to positions, not names.
- Six stands and five delta briefings are ready; other pairs take one command.
- The providers' raw data prove the chain without network; GitHub DORA is an approximation.
- AI drafts only with the switch, in their own folder, never approved.

Appendix: commands at a glance

```
python -m testdata_generator --web                          # UI in the browser
python -m testdata_generator --scenario portfolio --output demo --seed 42
python -m testdata_generator --scenario portfolio --story escalation --stands 6 --output demo-esk
python -m testdata_generator --scenario portfolio --arts 3,4s,1 --event E9=off --output demo-3
python -m testdata_generator --scenario portfolio --ai-drafts --output demo-ki
python -m testdata_generator --scenario portfolio --continue demo --to-date 2027-01-15
python -m portfolio demo/portfolio.json                     # portfolio report
python -m portfolio --delta demo/snapshots/snapshot_05_*.json demo/snapshots/snapshot_06_*.json
python -m sources fetch --kind slo --config demo/sources/alpha/slo_sources.json --output fetched_slo.json
```